

COMPUTER NETWORKS PROJECT

GROUP - 7

CS22BTECH11050 - Rishitha Surineni
CS22BTECH11046 - Paruvada Yasaswini
CS22BTECH11024 - Gugulothu Sai Keerthi
CS22BTECH11056 - Somalaraju Bhavya Shloka
CS22BTECH11006 - Annam Deddeepya Purnima
ES22BTECH11005 - Jaya Karthik

1 Implementation

- We utilized the concept of multi-threading to implement a load balancer (a host) that distributes client requests among available servers, prioritizing those with less load.
- Client requests are forwarded to a server based on the least response time (i.e., the server that responds the fastest).
- Initially, the response times of all servers are set to 0. When a client request is made, the load balancer checks for the server with the minimum response time.
- The load balancer selects a response time by calculating an average of the most recent 5 response times in the lists for each server, using a sliding window of size 5 (i.e., it considers and averages them).
- It forwards the client's request to the chosen server, receives the response from the server, and updates the response times list by adding a new entry in the response times list of the respective server.

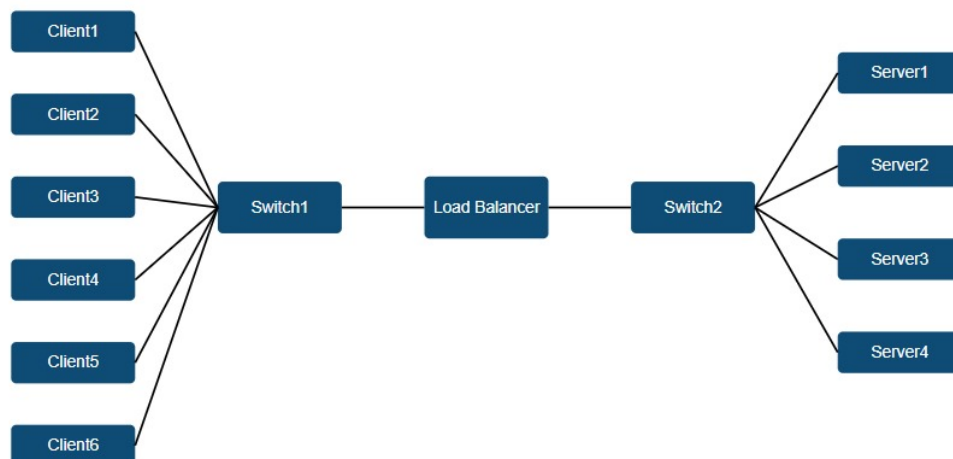


Figure 1: System Block Diagram

2 Algorithm

Input: Client Request, Response Time List for Each Server

Output: Request Assigned to the Best Server

Begin

1. **Initialize:**

- response_Times = Dictionary to store response times for each server.
- Initialize all lists with 0.

2. **Handle Client Requests:** Whenever a new client request comes in:

- Select the server with the lowest response time using:
- `chosen_server = argmin([average(last_5_times) for each server])`
- Assign the request to `chosen_server`.
- Create a new thread to handle the forwarding of the request to the server.

3. **Forward Request:**

- Forward the client request to the `chosen_server`.
- Wait for the server's response on the particular thread and, after receiving, add the response time to the response time list of the server and finally relay the response back to the client.

End

3 Code Details

Concept	Where in Code	Purpose
Sliding Window Average	<code>get_server_average_response_time()</code>	Uses the last <code>window_size</code> response times to compute the average for fairness (for each server).
Dynamic Server Selection	<code>get_next_server()</code>	Chooses the best-performing server dynamically based on response times.
Forwarding Client Request	<code>handle_client()</code>	Handles a particular client request and the responses between clients and servers.
Updating Server Response Times for Client Requests	<code>measure_response_time()</code>	Handles forwarding of client requests to the chosen server, measures the time taken for response, and updates the list storing response times for the respective server.

Table 1: Code Details for Load Balancer Implementation



Figure 2: Flow diagram

4 Execution

1. Execute the custom topology file (Topology.py)

```
sudo python Topology.py
```

Note: If the topology (2 clients - 3 servers - 1 load balancer) is needed to be changed , add appropriate statements like adding hosts, add appropriate links to switches.

e.g:

```
server1 = self.addHost('server1')
client1 = self.addHost('c1')
self.addLink(client1, client\_switch)
```

2. When the mininet environment has started, open all the hosts(load balancer, clients, servers) using the command below

```
xterm h1 c1 c2 server1 server2 server3 (for 2 clients and 3 servers)
```

3. Execute respective server, client, load balancer files in h1, c1 , c2, server1,.. And add delays to see the difference between load balancing techniques.

```
tc qdisc add dev server1-eth0 root netem delay <x ms>
python client.py (in c1 and c2)
python server1.py (in server1)
python server2.py (in server1)
python server3.py (in server1)
python c1.py (in c1 - load balancer)
```

5 Evaluation

The performance of the load balancer can be evaluated using two key metrics: **Average Latency** and **Throughput**.

1. **Average Latency**

Average latency is calculated by measuring the time taken for each request-response cycle and then averaging these times. This metric reflects how quickly the load balancer is able to process requests and receive responses from the servers.

The formula for Average Latency is:

$$\text{Average Latency (ms)} = \frac{\sum \text{latencies of all requests}}{\text{number of requests}}$$

Where:

- $\sum$ latencies of all requests is the sum of the time taken for each request-response cycle.
- number of requests is the total number of requests processed by the system.

2. **Throughput**

Throughput measures how many requests the load balancer can process per second. It is calculated by dividing the total number of requests by the total elapsed time (the time between sending a request and receiving the response).

The formula for Throughput is:

$$\text{Throughput (requests/sec)} = \frac{\text{Total Requests}}{\text{Total elapsed time}}$$

Where:

- Total Requests is the total number of client requests processed.
- Total elapsed time is the total time taken for the entire set of requests to be processed (from sending requests to receiving responses).

These two metrics, Average Latency and Throughput, provide an insight into the efficiency and performance of the load balancing system.

Latency

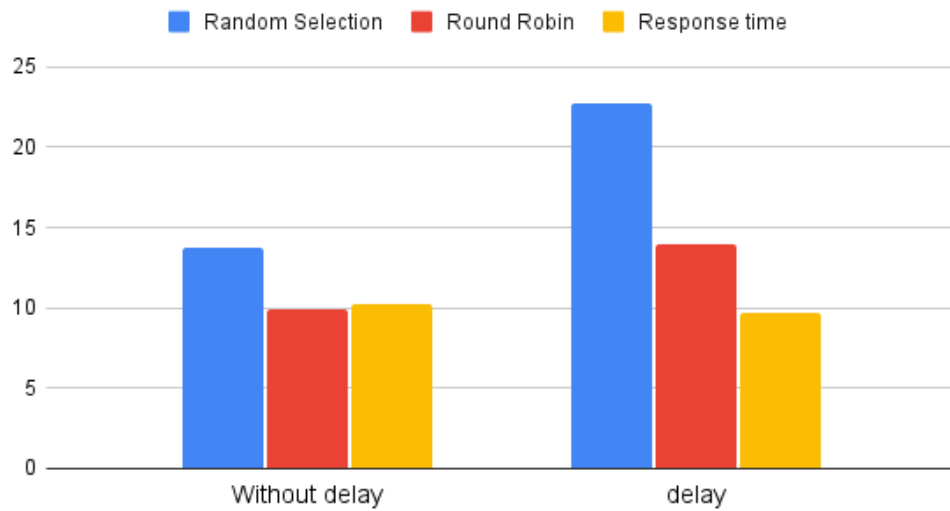


Figure 3: 2 clients - 3 servers with and without delays

Throughput

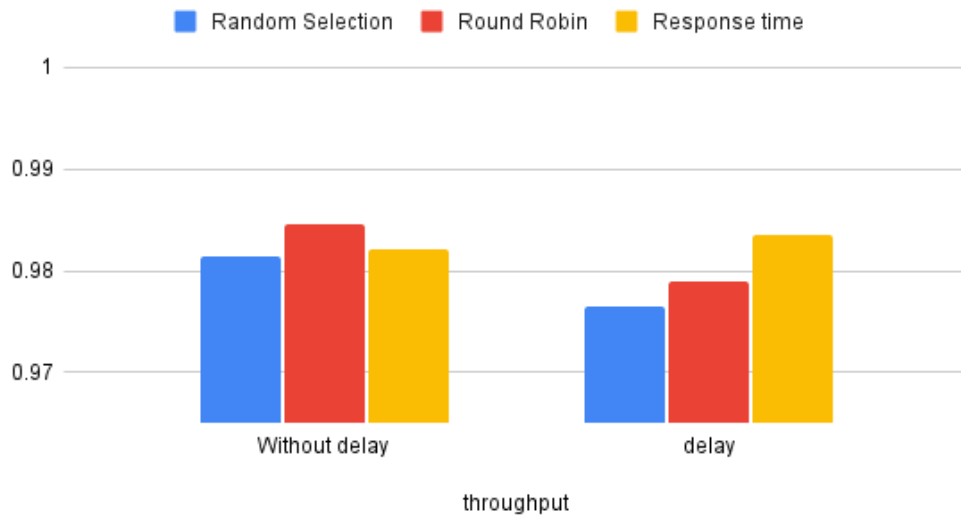


Figure 4: 2 clients - 3 servers with and without delays

Conclusion about Fig 3 and Fig 4 : When a delay is imposed, the round-robin algorithm does not account for it and continues to distribute requests in a predetermined order. In contrast, the least response time algorithm changes dynamically by deprioritizing delayed servers, ensuring that requests are sent to servers with faster response times, so improving overall performance.

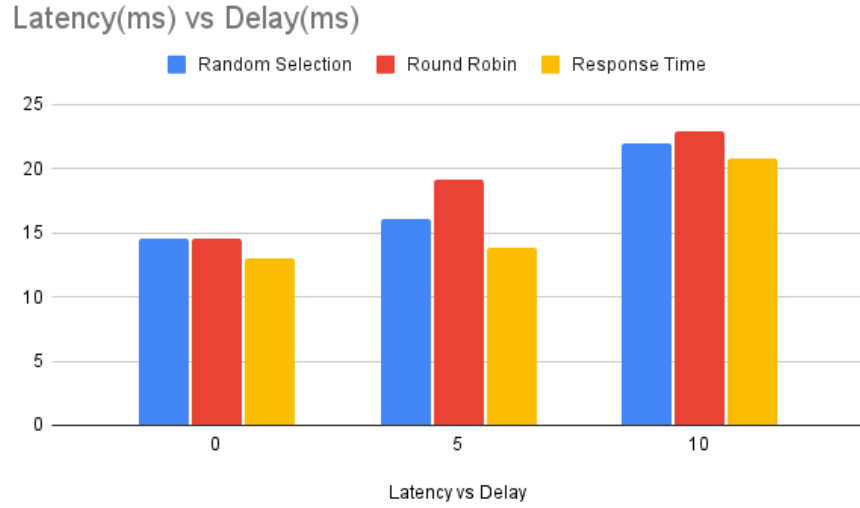


Figure 5: 6 clients - 4 servers — Latency with varying Delays in a server

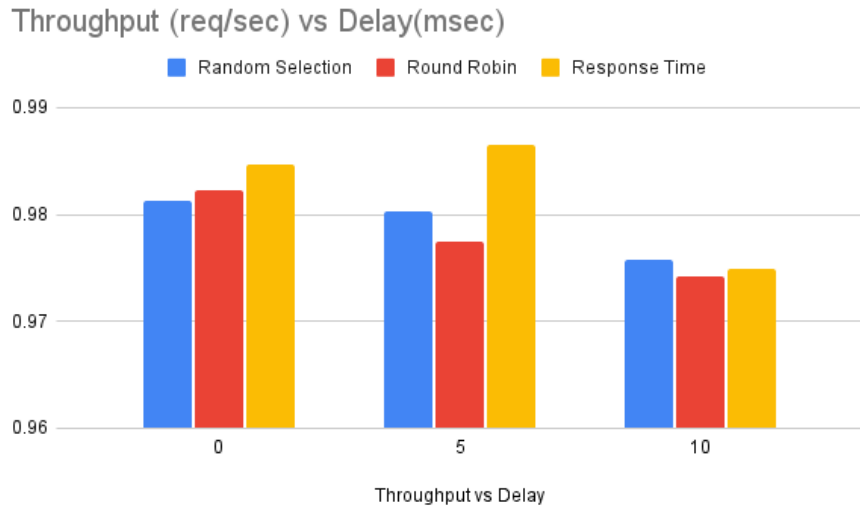


Figure 6: 6 clients - 4 servers — throughput with varying Delays in a server

Conclusion about Fig 5 and Fig 6

By increasing the delay values (0ms , 5 ms, 10ms) for server it can be seen that the Throughput obtained is higher and Latency obtained is lower for Response time algorithm when compared to Round Robin and Random Selection, it is because in response time algorithm the server with minimum response time is selected which results in not selecting the server with delay. But in case of round robin or random selection some of the requests are forwarded to server with delay resulting in more latency and less throughput

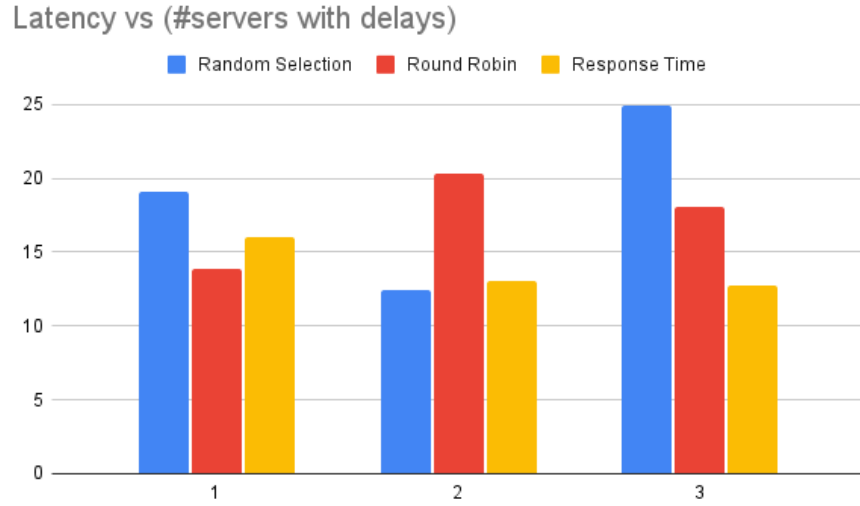


Figure 7: 6 clients - 4 servers — Latency with varying no. of servers having delays()

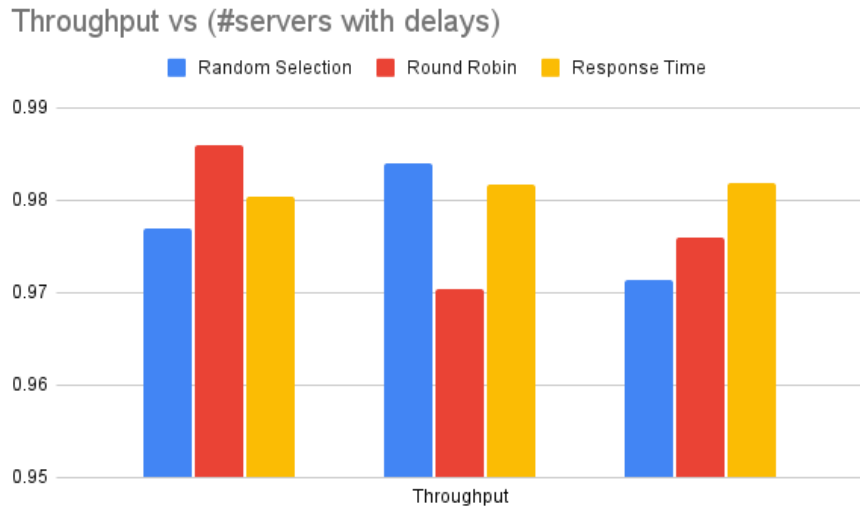


Figure 8: 6 clients - 4 servers — throughput with varying no. of servers having delays()

Conclusion about Fig 7 and Fig 8

when we increase the number of servers with delays we can observe the response time algorithm performs better than round-robin and random selection, this is because more number of requests will be sent to servers with delay in case of random and round-robin